OPERATING SYSTEMS CASE STUDY (BCSE303L) - FINAL REPORT

Predictive Deadlock Prevention Using Contention Analysis and Score Forecasting

Real-thread experiments, early warning, lock ordering, victim recovery, and evaluation

Team: Midhun Gereketi (frontend, visualisation, integration) · P. Satya Mani Shraavanth (threaded experiments and API) · M. Lakshman Sai (analysis, scoring, evaluation).

Live app: https://predictive-deadlock-lab.vercel.app/ · **Source:** https://github.com/midhungereketi/predictive-deadlock-lab

Abstract

The project creates controlled deadlocks with real operating-system threads and studies how to observe, warn about, prevent and recover from them. In the original two-thread experiment T1 takes lock A then B while T2 takes B then A; a Wait-For Graph (WFG) confirms the cycle and a weighted 0-100 risk score warns before it appears. Protected mode learns one global lock order from a cycle-free probe. A configurable N-thread ring (2-32 threads on N locks) forms one cycle through every thread; a supervisor detects it and recovers by aborting a victim thread, while protected mode uses a global order and always completes. The dashboard replays each run and its Analysis and Compare views show when and why the score warned and how the modes differ over repeated trials and thread counts.

In the saved set (15 Oct 2026) 10/10 vulnerable two-thread runs deadlocked and 10/10 protected runs completed. A new collection (26 Oct 2026) reproduced this (10/10, 10/10) and added a scaling study: 50/50 vulnerable ring runs deadlocked with a cycle through all N threads and 50/50 recovered with one aborted thread; 50/50 protected runs completed. **The warning is a documented heuristic evaluated in a controlled experiment; the project does not claim to predict deadlocks in arbitrary programs.**

1. Problem and objectives

A deadlock needs mutual exclusion, hold-and-wait, no preemption and circular wait at the same time; opposite lock orders make circular wait possible. The objectives are to reproduce circular wait with real threads, detect an actual cycle from observed events, warn before it using only current information and measure how early, prevent it with a common lock order and recover from it by preempting a victim, and keep every measurement inspectable and repeatable.

2. Architecture

The browser dashboard (Lab, Analysis, Compare, Method) calls a FastAPI backend that also serves the dashboard, so the same code runs locally and on Vercel. The backend keeps no mutable state between requests: each request runs its own bounded experiment and builds its own scorer. Experiments emit timestamped lock events (high-resolution monotonic clock); the analysis builds the WFG, risk score, trend and heatmap and returns trial JSON or results to the browser.

Component	Implementation
Two-thread experiments	member2_backend/vulnerable_experiment.py, protected_experiment.py, adaptive_policy.py, experiment_runner.py

Component	Implementation
Lock tracking and cycle check	member2_backend/lock_manager.py (bounded and cancellable waits)
N-thread ring and recovery	member2_backend/multi_thread_experiment.py
WFG, score, trend, heatmap	member3_analysis/graph.py, threat_scorer.py, trend_analyzer.py, heatmap.py
Request-scoped analysis API	member3_analysis/api_routes.py, comparator.py
Dashboard and views	frontend/index.html, app.js, styles.css, scoring.js

3. Algorithms

Wait-For Graph and cycle detection

An edge Ti → Tj exists only while Ti waits for a lock owned by Tj. Each thread waits for at most one lock, so each node has at most one outgoing edge; following edges from each node finds any cycle in linear time. With single-instance locks a cycle is a confirmed deadlock.

Risk score (early-warning heuristic)

```
Score = round_half_up(w_b·B + w_w·W + w_c·C + w_r·R), where B = blocked threads / 2 × 100, W = longest wait / 800 ms × 100 (capped), C = contention events / 5 × 100 (capped), and R = 100 when A and B are held by different threads, 40 when one thread holds both, 20 when one lock is held. Defaults: 0.35, 0.30, 0.20, 0.15; threshold 70.
```

Two rules apply: when different threads hold A and B and one waits, the score is at least 72; a confirmed cycle sets it to at least 96. The score uses only the current snapshot. **Weights are request-scoped:** each visitor's weights are applied in their browser and sent with the requests they start; they are stored only in that browser if the visitor saves them, never on the server, so visitors cannot affect each other. Python and the browser implement the same formula with round-half-up and are tested against the same shared cases.

Adaptive protected policy (two threads)

A probe lets T1 hold A while waiting for B held by T2 and stops before T2 requests A, so no cycle forms. The observed held → requested pair yields the order A → B; both workers sort their requests into that order before taking any lock, so no thread can hold B while waiting for A.

N-thread ring with victim recovery

Thread Ti needs Ri and R(i mod N)+1. Vulnerable: every thread takes its own lock, all meet at a barrier, then each requests the next lock, giving one cycle through all N threads. A supervisor polls a consistent snapshot every millisecond, detects the cycle and aborts the highest-numbered thread by cancelling its blocked acquire; its lock is released and the others finish one after another. Protected: each thread takes its lower-numbered lock first, so the last thread requests R1 before RN and circular wait cannot form.

4. Dashboard and workflow

Lab runs the two-thread trial or an N-thread ring and replays it; Analysis explains when and why the score warned and re-scores the run as weights change; Compare runs both modes, repeated trials and the scaling test, with CSV export.

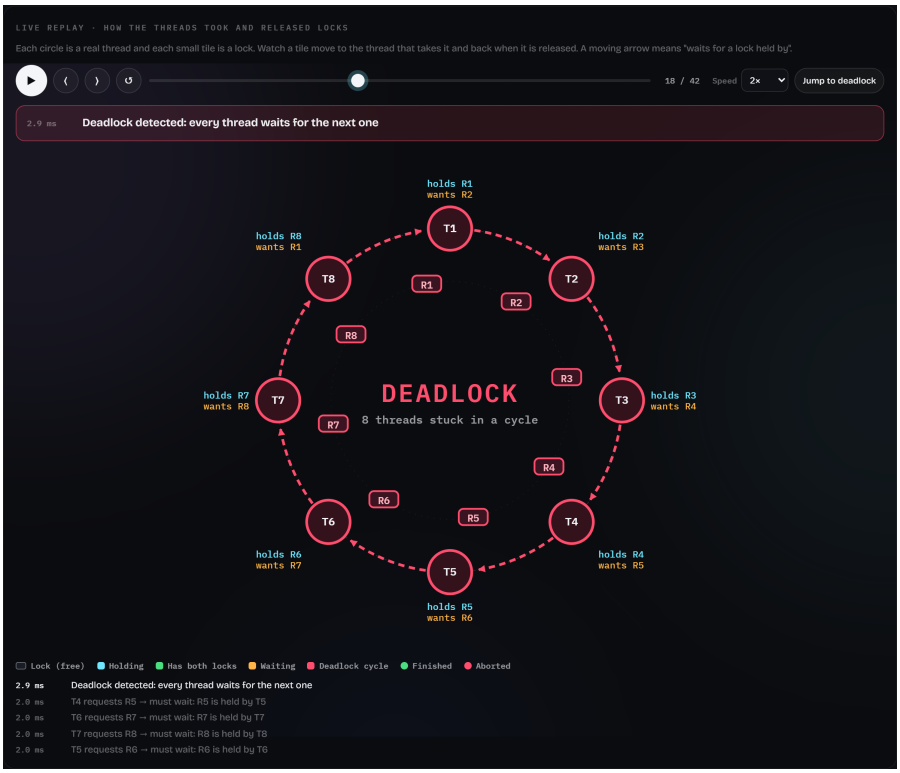


Figure 1. Replay of a real 8-thread vulnerable run at the moment the supervisor detected the cycle through all 8 threads.

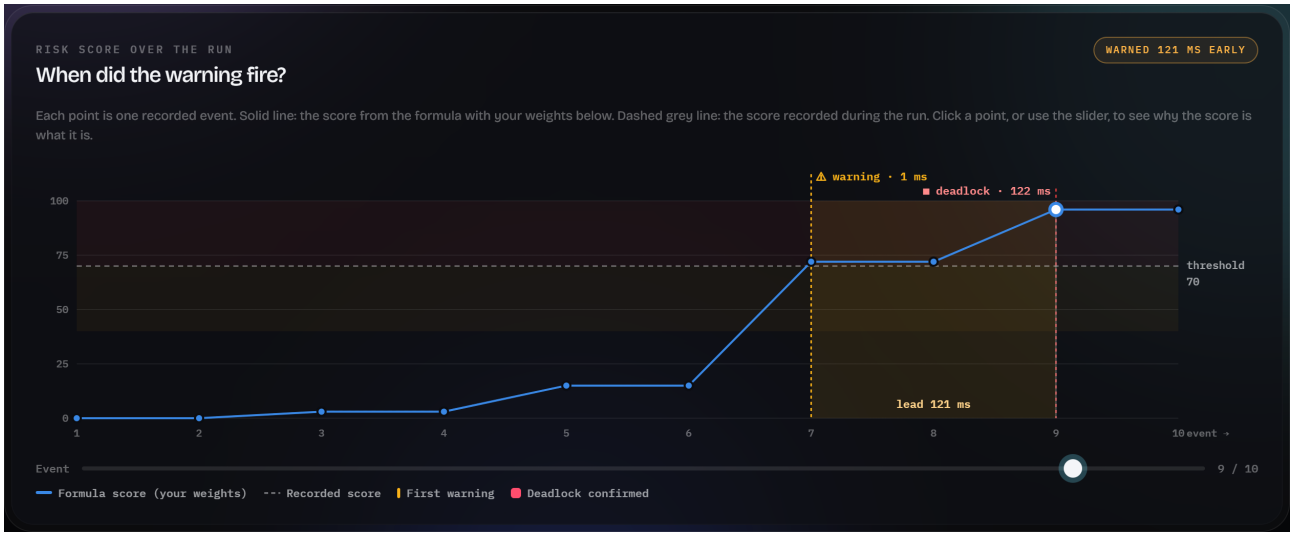


Figure 2. Analysis view of a live two-thread trial: the warning fired 121 ms before the deadlock was confirmed.

5. Evaluation protocol

- **Two-thread:** 10 trials per mode, 3 s cap per trial, 120 ms observation window before the conflicting request. Deadlock rate = confirmed-cycle trials / trials; completion rate = both workers finished / trials; warning lead = first pre-cycle score ≥ threshold to first cycle; detection latency = first cycle to supervisor notification. Failed runs count as inconclusive.

- **Scaling:** sizes 2, 4, 8, 16, 32; 10 trials per size and mode, modes alternating. Vulnerable runtime = start to detection, victim abort and all other threads finishing; protected runtime = start to every thread completing. Every raw run is saved with the summary.

- **Separation of data:** the saved set (docs/results.json, 15 Oct 2026) is kept unchanged; the new sets are in docs/results_2026-10-26_two_thread.json and docs/results_2026-10-26_scaling.json.

6. Results

6.1 Two-thread experiment: saved set and new set

Metric	Saved · vuln.	Saved · prot.	New · vuln.	New · prot.
Trials	10	10	10	10
Deadlocked / completed	10 / 0	0 / 10	10 / 0	0 / 10
Inconclusive	0	0	0	0
Mean runtime	813.6 ms	104.6 ms	813.4 ms	103.9 ms
Mean peak wait	809 ms	28.9 ms	808.4 ms	29.4 ms
Mean peak score	96	72	96	72
Warning lead before cycle	120.4 ms	n/a	121.6 ms	n/a
Detection latency	4.6 ms	n/a	3.1 ms	n/a

The two sets agree on every outcome. Small times are not directly comparable: the saved set used time.monotonic (about 15.6 ms ticks on Windows) and the new set uses time.perf_counter. The score's rounding changed from banker's rounding to round-half-up; peak scores are fixed by the 72/96 rules and are unaffected. Protected runs reach 72 during the safe probe, before the policy applies.

6.2 N-thread scaling (new set, 26 Oct 2026)

Threads	Deadlocked	Full cycle	Recovered	Detection mean (max)	Detection + recovery	Protected done	Completion time
2	10/10	10	10	1.81 (2.51) ms	45.4 ± 2.6 [39–49] ms	10/10	61.9 ± 0.9 [61–63] ms
4	10/10	10	10	1.97 (2.19) ms	107.2 ± 2.0 [105–111] ms	10/10	68.6 ± 19.1 [61–123] ms
8	10/10	10	10	2.49 (3.04) ms	229.4 ± 2.7 [226–236] ms	10/10	93.9 ± 24.9 [63–124] ms
16	10/10	10	10	4.20 (6.50) ms	473.0 ± 4.5 [468–480] ms	10/10	92.3 ± 17.1 [64–125] ms
32	10/10	10	10	6.65 (8.00) ms	967.8 ± 3.6 [964–975] ms	10/10	95.6 ± 9.1 [70–100] ms

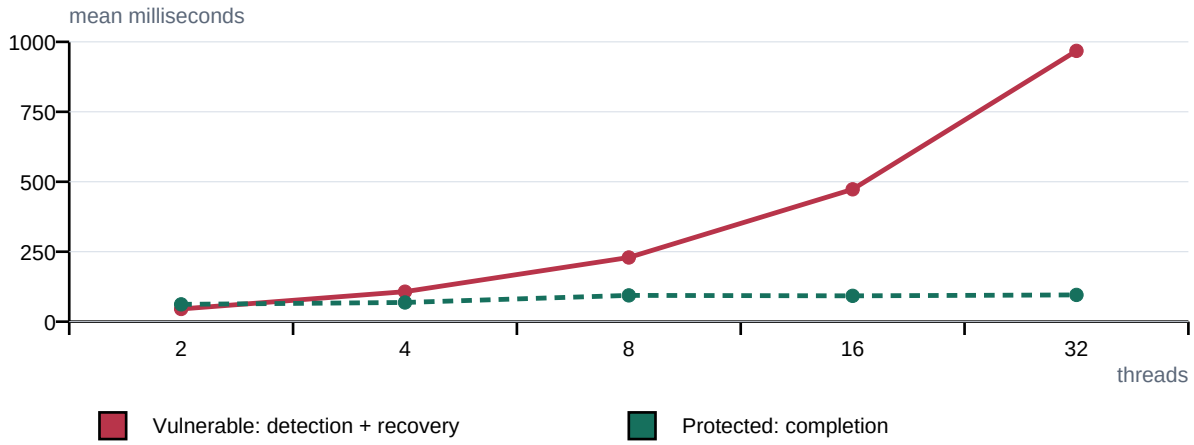


Figure 3. Mean run time by thread count (values from docs/results_2026-10-26_scaling.json). Vulnerable time includes detection and recovery; protected time is completion only.

Detection stayed within a few milliseconds; it grows with N, likely because more threads must reach the barrier and register their waits. Vulnerable time grows by roughly 30 ms per extra thread because, after the victim is aborted, the remaining threads finish one after another, each holding both locks for 30 ms. Protected time stays near 100 ms because non-neighbouring threads run in parallel. These are controlled runs of one workload on one machine, not a performance benchmark.

7. Limitations

- Controlled schedules (barriers, a 120 ms window) make cycles reproducible; the measured lead time describes this experiment only and is not general predictive accuracy.
- The two-thread policy learns its order before the workers start and does not reorder locks already held; the ring's victim choice (highest-numbered thread) ignores fairness and rollback cost.
- Python threads under the global interpreter lock, on one machine; timings vary with load. Score ceilings are designed for the two-thread case.
- Runs are bounded by timeouts and an in-process supervisor; this is a teaching prototype, not a process-isolated production service.

8. Literature review (2020-2025)

Work	Contribution	Relation to this project
Qin et al., PLDI 2020 [1]	Study of 170 real Rust bugs, including blocking bugs; static detectors	Lock-order deadlocks persist even in memory-safe languages
Cai, Meng & Palsberg, ICSE 2020 [2]	AirLock: cycle search over a small lock reachability graph, ~3.5% overhead	Our supervisor also searches a small graph; we confirm cycles at runtime
Liu et al., ASPLOS 2021 [3]	GCatch detects and GFix repairs Go blocking bugs	Detect then repair, versus our detect then recover
Tunç et al., PLDI 2023 [4]	Sound and complete prediction is intractable in general; linear-time algorithms for sync-preserving deadlocks	Why our score is presented as a heuristic warning
Qin et al., IEEE TSE 2024 [5]	100 Rust concurrency bugs; five static detectors (lockbud)	Supports lock ordering as prevention
van den Heuvel, Sulzmann & Thiemann, arXiv 2025 [6]	Partial orders filter false-positive deadlock warnings	Why false alarms must be reported honestly

9. Testing

Python tests cover the experiments, API, the ring at 2, 5, 12 and 32 threads, shared-case scoring consistency and scoring isolation (sequential, concurrent and real-trial checks that one visitor's weights never change another's results). Node tests cover the trial contract and the browser scoring formula. Browser tests (Playwright with a real server) cover tab switching, live weight tuning and its saved/temporary states, per-visitor privacy including backend scores, running both modes, repeated trials, scaling, injected backend failures and CSV exports.

10. Team roles

- **Midhun Gereketi:** dashboard, replay and visualisation, Analysis and Compare views, trial contract, integration, deployment, documentation.
- **P. Satya Mani Shraavanth:** threaded experiments, lock manager, protected policy, N-thread ring and recovery, REST API.
- **M. Lakshman Sai:** Wait-For Graph, risk score, trend and heatmap analysis, repeated-trial comparison and evaluation.

11. Reproduction

```
pip install -r requirements-dev.txt
python -m pytest member2_backend/tests member3_analysis/tests tests/e2e
node --test frontend/tests/trial-data.test.js frontend/tests/scoring.test.js
python -m uvicorn member2_backend.api:app --host 127.0.0.1 --port 8000
python scripts/collect_scaling_results.py --sizes 2 4 8 16 32 --trials 10 --output docs/<new>.json
```

12. Conclusion

Real threads reproduce circular wait on demand from 2 to 32 threads; a Wait-For Graph confirms it; the score warns before it in the staged two-thread case; a global lock order prevents it; and aborting one victim recovers from it. Every claim above is backed by saved, repeatable measurements, and the stated limits define where the results apply.

References

[1] B. Qin, Y. Chen, Z. Yu, L. Song, Y. Zhang. Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs. PLDI 2020. pldi20.sigplan.org

[2] Y. Cai, R. Meng, J. Palsberg. Low-Overhead Deadlock Prediction. ICSE 2020, pp. 1298-1309. doi:10.1145/3377811.3380367

[3] Z. Liu, S. Zhu, B. Qin, H. Chen, L. Song. Automatically Detecting and Fixing Concurrency Bugs in Go Software Systems. ASPLOS 2021. github.com/system-pclub/GCatch

[4] H. C. Tunç, U. Mathur, A. Pavlogiannis, M. Viswanathan. Sound Dynamic Deadlock Prediction in Linear Time. PLDI 2023. doi:10.1145/3591291

[5] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, Y. Zhang. Understanding and Detecting Real-World Safety Issues in Rust. IEEE Transactions on Software Engineering, 2024. doi:10.1109/TSE.2024.3380393

[6] B. van den Heuvel, M. Sulzmann, P. Thiemann. Partial Orders for Precise and Efficient Dynamic Deadlock Prediction. arXiv:2502.20070, 2025 (preprint).